AI Loan Eligibility Checker

Project Documentation

AI-Powered BFSI Web Platform for Loan Eligibility, Credit Analysis, EMI Planning and Financial Guidance Demonstration

Project Description

FinWise AI – Smart Loan & Financial Assistant is an AI-powered web-based financial assistance platform designed to help users understand their financial position and make informed decisions related to loans and personal finance. The platform combines multiple financial tools into one interface, including a loan eligibility checker, credit score analyzer, EMI calculator, AI financial tips, and a dashboard. It provides users with a simple and interactive way to enter their financial details and receive useful financial insights.

The Loan Eligibility Checker evaluates details such as age, employment type, monthly income, existing EMIs, credit score, requested loan amount, loan tenure, employment duration, and estimated expenses. The Credit Score Analyzer helps users understand their credit score by analyzing factors such as credit utilization, payment history, recent credit inquiries, active loans, and credit cards. The EMI Calculator calculates the estimated monthly EMI, total interest, total payable amount, and amortization schedule based on the loan amount, interest rate, and tenure.

The platform also includes an AI Financial Tips module that allows users to ask general financial questions and receive educational guidance. The project uses HTML, CSS, and JavaScript for the frontend, with a Flask-based backend architecture and Anthropic Claude API integration described in the project configuration. A dashboard provides a summary of the user's assessment, including estimated eligible loan amount, credit tier, EMI burden, risk level, and recent activity. Overall, FinWise AI demonstrates the integration of web development, financial calculations, AI assistance, and data management into a single user-friendly financial support platform.

Project Scenarios

Scenario 1: Loan Eligibility Assessment

A user enters age, employment type, monthly net income, existing monthly EMIs, credit score, requested loan amount, tenure, employment duration and estimated expenses. The system validates the input and calculates an estimated borrowing capacity, EMI burden, eligibility status and risk level.

Scenario 2: Credit Score Review

A user enters a credit score and supporting credit information such as card utilization, payment history, inquiries, active loans and credit cards. The analyzer displays a score category and identifies positive factors and improvement areas.

Scenario 3: EMI Planning

A user enters principal, annual interest rate and tenure. The calculator estimates the monthly EMI, total interest, total payable amount and an amortization view.

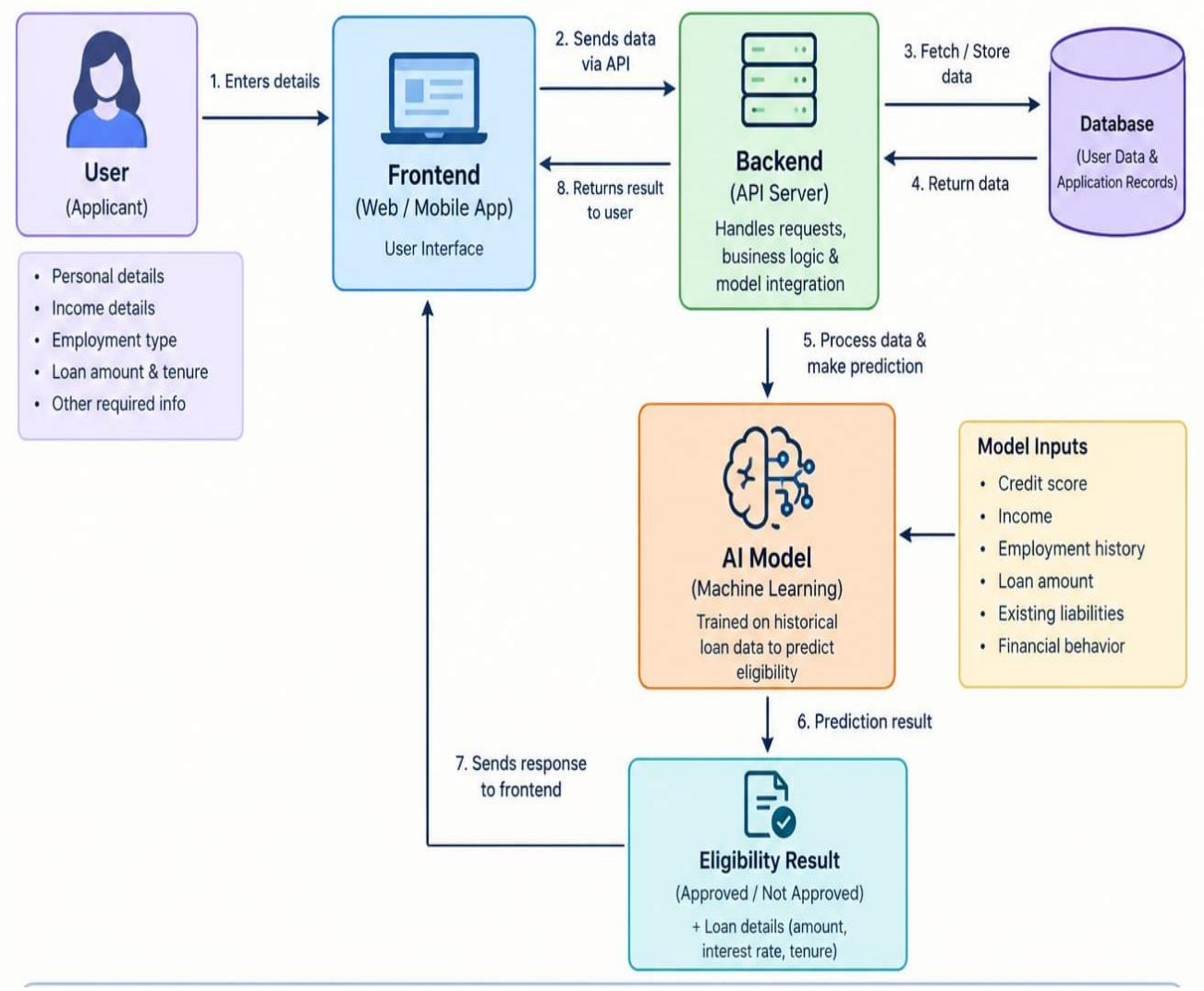
Scenario 4: AI Financial Guidance

A user asks a question using quick prompts or free text. The application sends the query to the documented AI endpoint when available and uses a client-side educational fallback when the API request fails.

Scenario 5: Dashboard Review

After an assessment, the dashboard summarizes eligible amount, credit tier, EMI burden, risk level and recent activity so that the user can review the latest results.

System Architecture :



High-Level Flow

User → Web Interface → Input Validation → Financial Calculation / AI Request → Result → Dashboard / Activity

Project Workflow

Milestone 1: Requirement Analysis and Architecture

- Activity 1.1: Identify the financial problem and define the major user requirements.
- Activity 1.2**:** Select the required modules: loan eligibility, credit score analyzer, EMI calculator, AI financial tips, and dashboard.
- Activity 1.3**:** Define the interaction between frontend, backend, AI service, and data storage.
- Activity 1.4**:** Prepare the project configuration for Flask, Anthropic Claude API, and optional Google Sheets integration.

Milestone 2: Core Financial Features

- Activity 2.1: Implement loan eligibility input collection and financial calculations.
- Activity 2.2**:** Implement credit score analysis with risk levels and improvement factors.
- Activity 2.3**:** Implement EMI calculation, total interest calculation, and amortization schedule.

Milestone 3: AI Integration and Dashboard

- Activity 3.1**:** Integrate the AI chat functionality and quick financial prompts.
- Activity 3.2**:** Implement client-side fallback responses when the AI service is unavailable.
- Activity 3.3: Connect assessment results with dashboard metrics and recent activity.

Milestone 4: Frontend Development

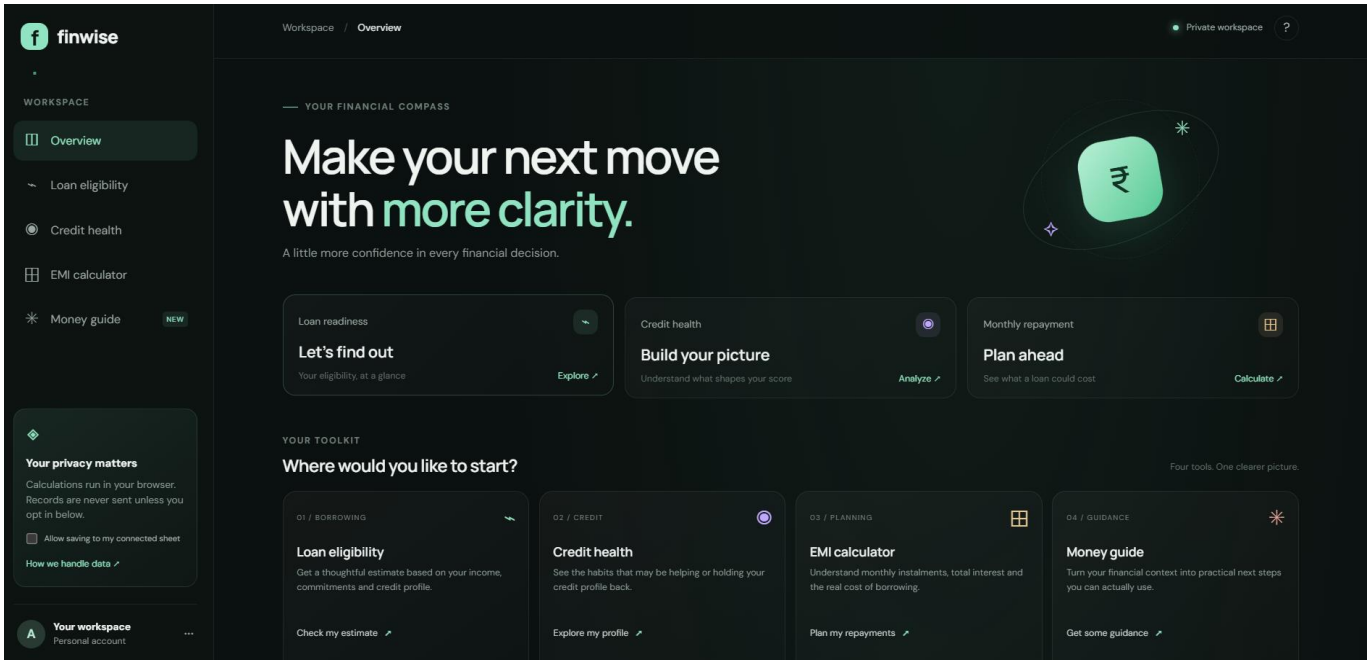
- Activity 4.1: Develop responsive HTML pages, forms, navigation, and result sections.
- Activity 4.2**:** Apply the glassmorphism design, cards, buttons, indicators, and responsive layouts.

Milestone 5: Testing and Deployment

- Activity 5.1: Test input validation, financial calculations, and different user scenarios.
- Activity 5.2: Test API communication, error handling, and AI fallback functionality.
- Activity 5.3: Prepare the application for local Flask execution and optional cloud deployment.

Module 1 – Home / Landing Page

The home page introduces FinWise AI and directs the user toward the main financial tools. The navigation structure contains the Home, Eligibility, Credit Analyzer, EMI Calculator, AI Tips and Dashboard views.



Main Elements

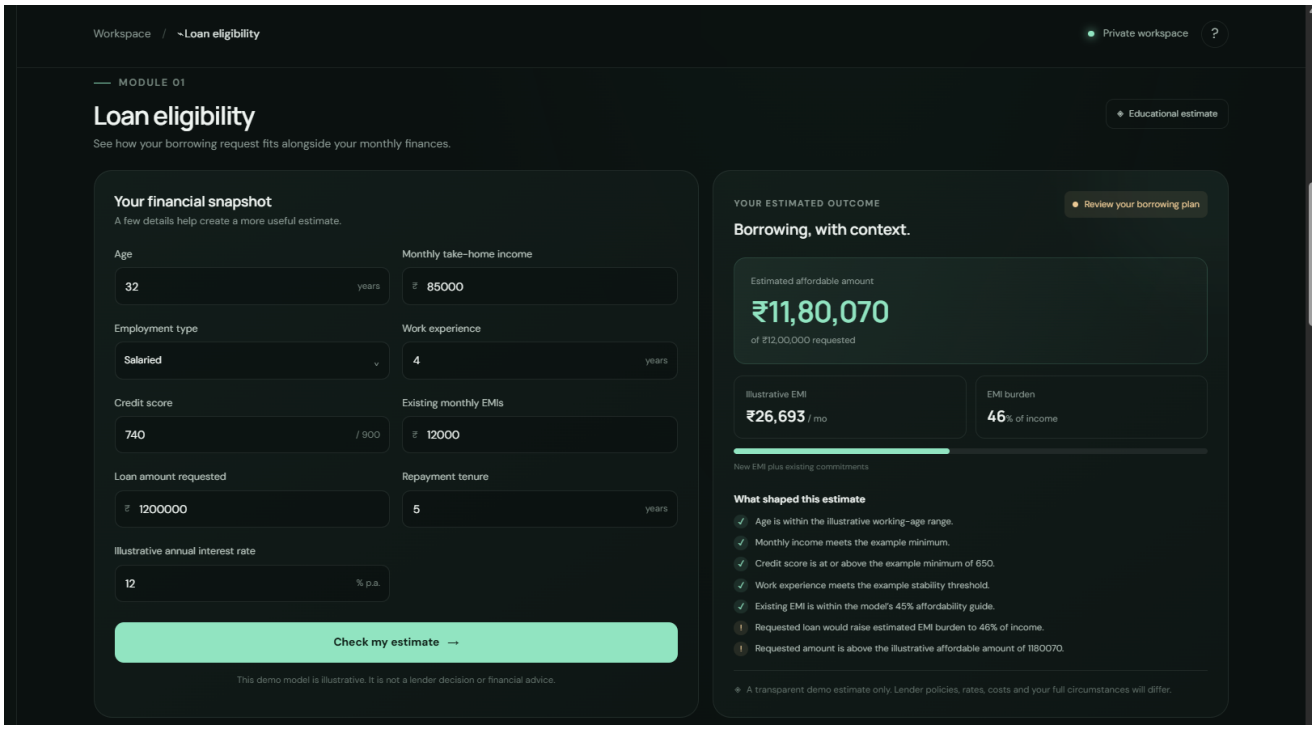
- Project introduction and value proposition.
- Primary Get Started action.
- Financial tool navigation.
- Feature cards explaining the major services.
- Workflow section describing data entry, AI processing and personalized insights.

User Flow

Home → Get Started → Select Financial Tool → Enter Details → Analyze → View Result

Module 2 – Loan Eligibility Checker

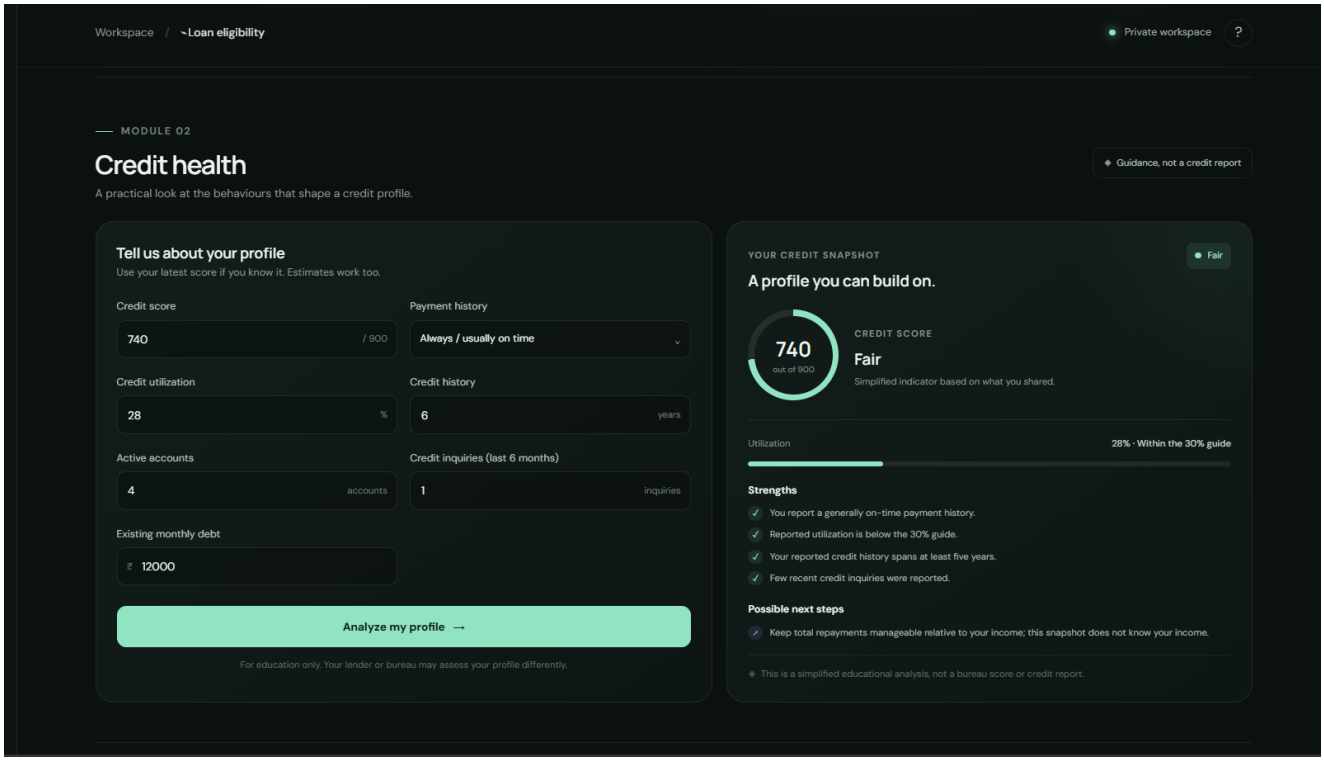
The Loan Eligibility Checker is the central assessment module. It collects personal, employment and financial inputs and calculates an estimated borrowing capacity using the application's rules.



Input	Output
Full Name	Eligibility Status
Age	Estimated Maximum Eligible Loan
Employment Type	Estimated EMI
Monthly Net Income	DTI/FOIR – style burden measure
Existing Monthly EMI’s	Risk level
Credit Score	Positive and Negative Factors
Requested Loan Amount	Suggestions for improvements
Loan Tenure	
Employment Duration	
Estimated Monthly Expenses	

Module 3 – Credit Score Analyzer

The Credit Score Analyzer presents the entered score on a 300–900 scale and classifies it into a project-defined category.



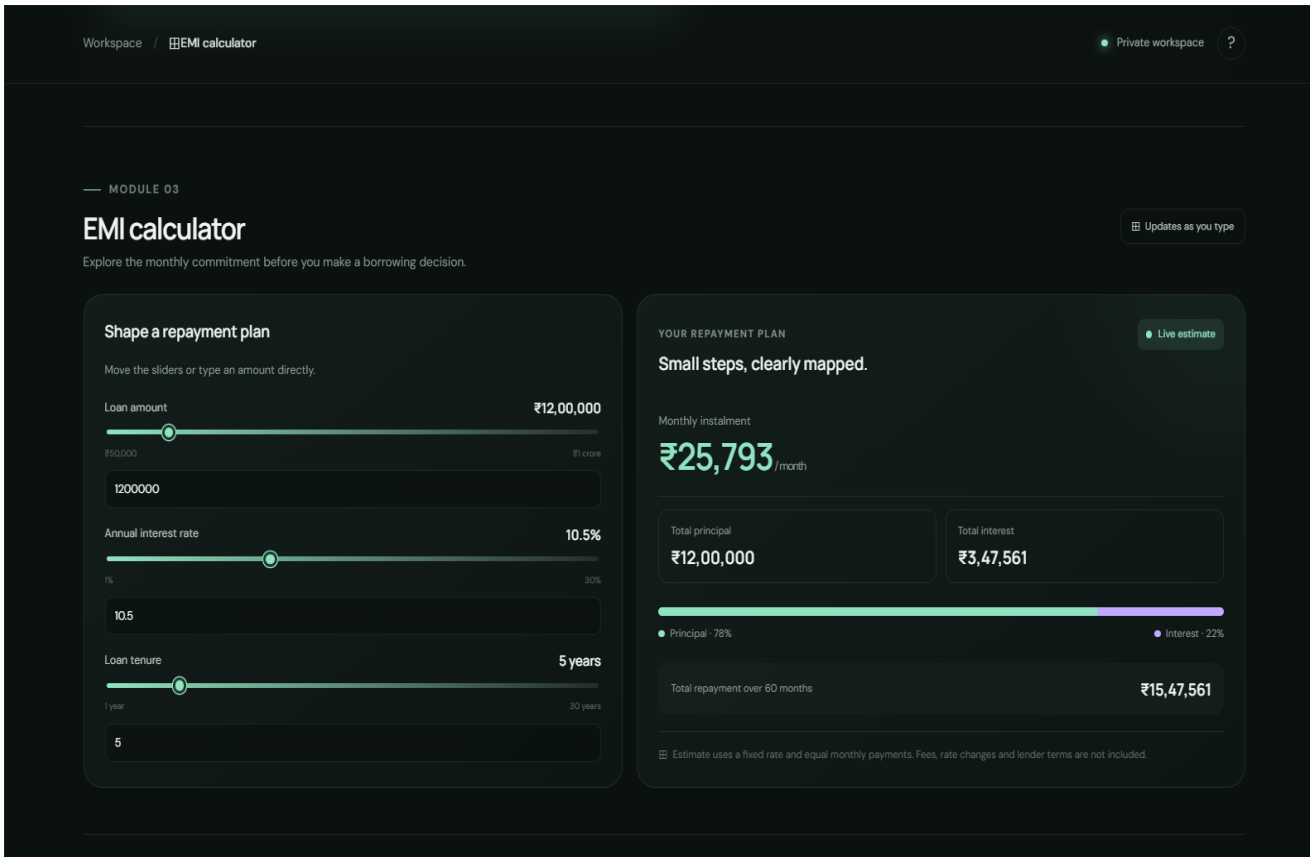
Score Range	Category
800–900	Excellent
740–799	Very Good
670–739	Good
600–669	Fair
300–599	Poor

Analyzed Factors

- Credit card utilization
- Payment history
- Recent credit inquiries
- Active loans and credit cards
- Outstanding debt

Module 4 – EMI Calculator

The EMI Calculator estimates the fixed monthly installment for a loan using principal, annual interest rate and repayment tenure.



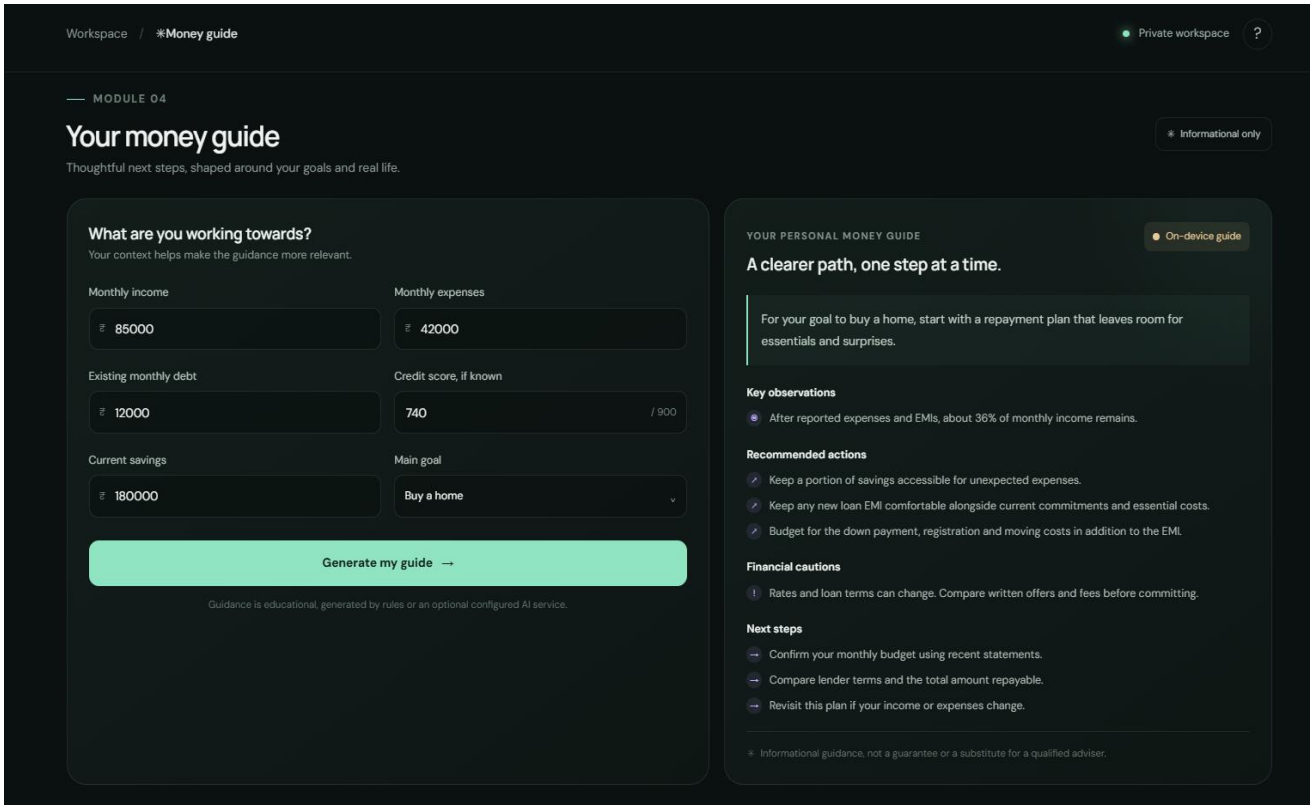
Formula

$$EMI = P \times R \times (1 + R)^N / ((1 + R)^N - 1)$$

Where P is the principal amount, R is the monthly interest rate expressed as a decimal, and N is the total number of monthly installments.

Module 5 – AI Financial Tips

The AI Financial Tips module provides a conversational interface for educational financial guidance. The frontend sends chat requests to the documented /api/chat endpoint and includes relevant user context such as name, income, credit score, existing EMI and eligibility status when available.



Quick Prompt Examples

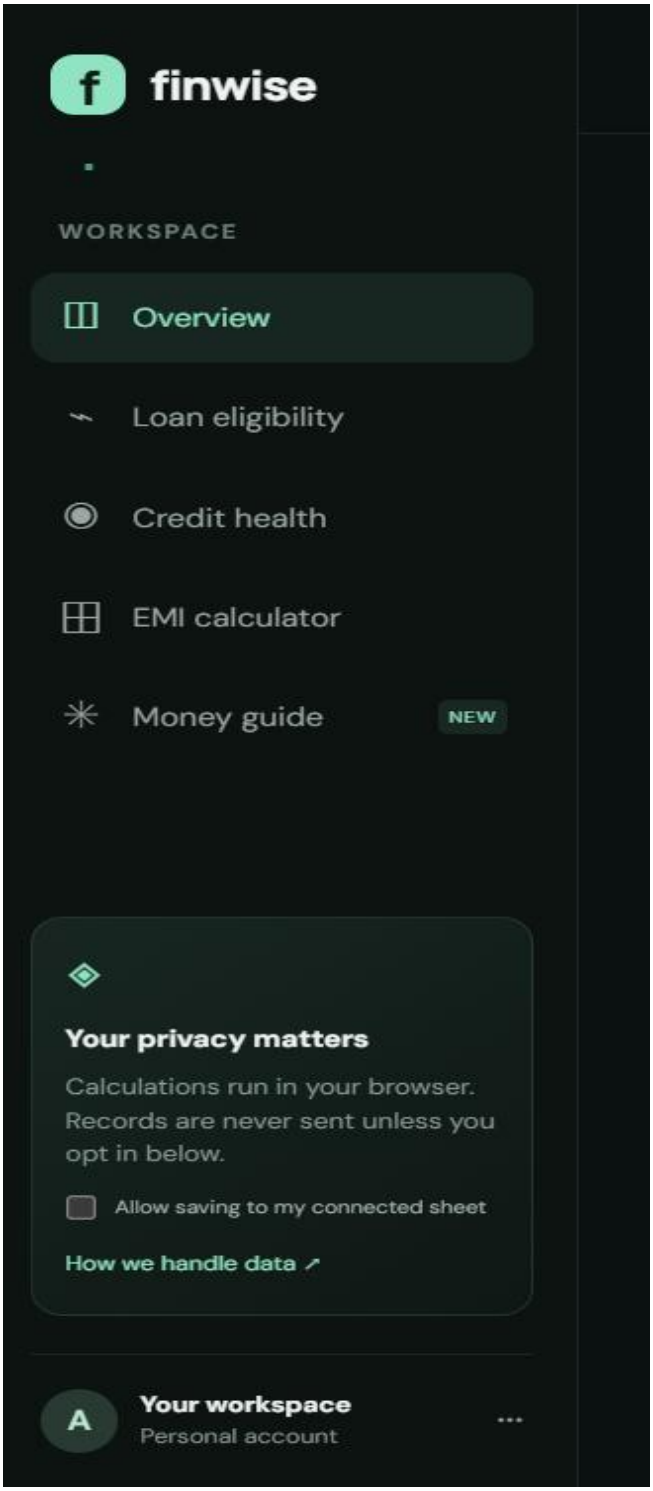
- How can I improve my credit score?
- Can I afford this loan?
- How can I reduce my EMI?
- Give me general financial guidance.

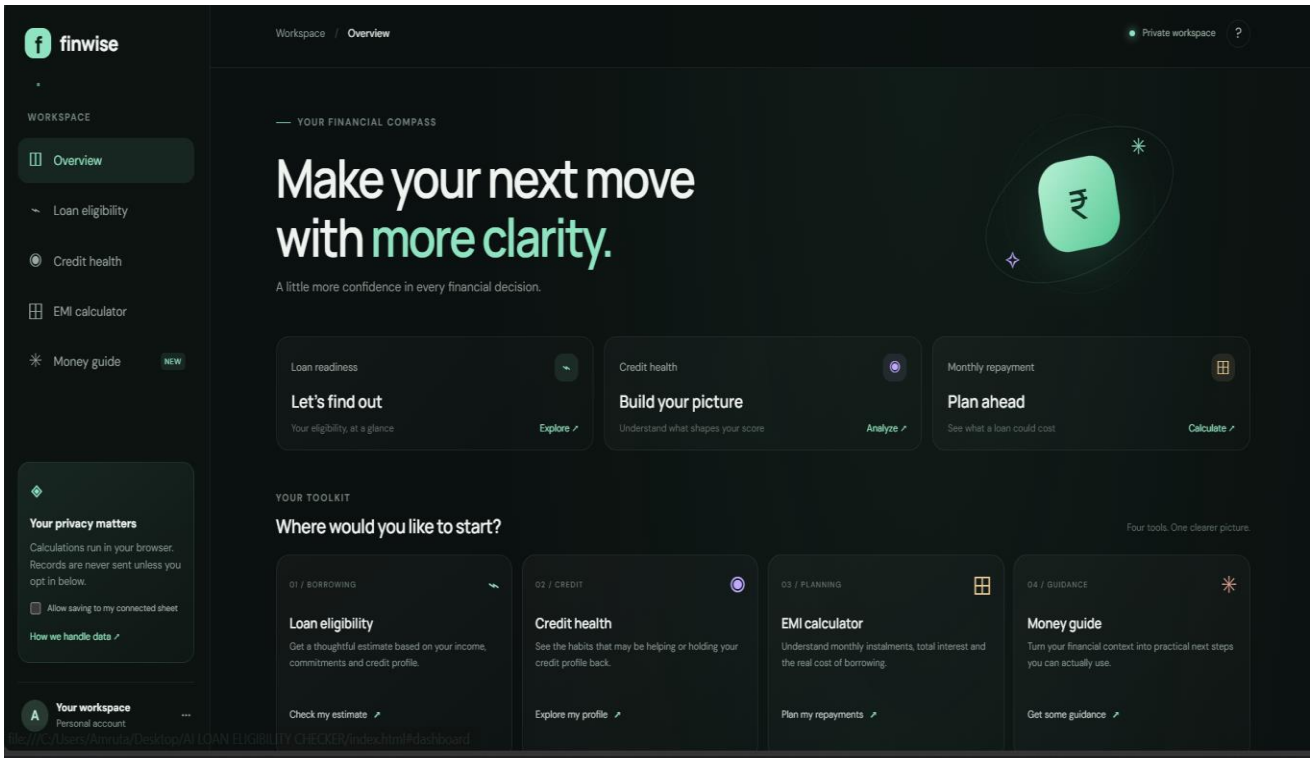
Fallback Mechanism

If the AI request is unavailable, the client-side fallback engine generates predefined educational responses for common financial topics. This allows the interface to remain usable for demonstration purposes even when an external AI service cannot be reached.

Module 6 – Dashboard

The dashboard gives the user a consolidated view of the latest assessment.





Dashboard Metrics

- Eligible loan amount
- Credit tier / credit score category
- Eligible EMI or EMI burden
- Risk level / financial health summary
- Recent activity
- Quick actions to start a new assessment or open financial tools

Data Flow and API Integration

Data Storage Endpoint

The frontend is configured to send assessment information to /api/store-user-data. The documented record includes generated user ID, name, age, employment type, income, existing EMI, credit score, requested loan amount, tenure, eligibility result and risk.

AI Chat Endpoint

The frontend sends conversational requests to /api/chat. The intended backend integration uses the Anthropic SDK and the model configured through environment variables.

Integration Flow

Assessment Form
↓
JavaScript Validation
↓
Eligibility / Credit / EMI Processing
↓
Result + Dashboard Update
↓
Optional /api/store-user-data
↓
Google Sheets or System Storage

AI Tips
↓
/api/chat
↓
Anthropic Claude API
↓
AI Response
↓
Chat Interface

Project Structure

```
finwise-ai/
├── index.html
├── style.css
├── script.js
├── backend/
│   ├── app.py            # referenced by README; not supplied
│   └── requirements.txt
├── config/
│   └── .env.example
└── README.md
```

- **index.html** – This is the main HTML file of the project. It defines the structure of the website, including the different pages, forms, navigation menus, financial tools, and dashboard elements.
- **style.css** – This file contains the styling and visual design of the website. It manages the layout, fonts, colors, buttons, cards, responsive design, and the overall glassmorphism-style appearance of the application.
- **script.js** – This file contains the main JavaScript functionality. It handles navigation, form validation, loan eligibility calculations, credit score analysis, EMI calculations, AI chat interaction, dashboard updates, and other interactive features.
- **backend/** – This folder is intended to contain the backend-related files required for server-side operations and API integration.
- **app.py** – The project structure references this file as the Flask backend application. It is intended to handle API requests, AI integration, and storage-related operations. The actual app.py source file was not included in the supplied project files.
- **requirements.txt** – Contains the Python packages required for the backend, such as Flask, Flask-CORS, Anthropic, Google authentication, and related dependencies.
- **config/** – This folder contains configuration-related files.
- **.env.example** – Provides a template for environment variables such as the port, Anthropic API key, AI model, Google Sheets ID, and Google credentials path.
- **README.md** – This file provides project documentation, including the project overview, features, technology stack, project structure, setup instructions, configuration details, and deployment information.

User Interface and Design

The interface uses a modern glassmorphism-inspired visual system with a dark deep-slate background, translucent cards, strong blue/cyan accents and status indicators. The CSS includes responsive navigation and layout behavior for smaller screens.

Design Principles

- Simple navigation between financial tools.
- Card-based presentation of calculations and results.
- Clear status indicators for risk and credit categories.
- Responsive layouts for desktop and mobile widths.
- Interactive sliders, buttons, forms and chat controls.
- Consistent typography and spacing for readability.

Responsive Behavior

The stylesheet contains responsive rules for navigation and content layout so that the application's main modules can adapt to different screen sizes.

Testing Strategy

Functional Testing

- Check required fields and invalid numeric values.
- Verify credit-score range handling from 300 to 900.
- Verify EMI results for different principals, rates and tenures.
- Verify eligibility outputs for different income and EMI combinations.
- Verify credit-factor badges for utilization, payment history and inquiries.
- Verify AI fallback behavior when the chat request fails.
- Verify dashboard updates after an assessment.

Boundary Testing

- Age below 18 and above 75.
- Income below the minimum configured threshold.
- Credit score below 300 or above 900.
- Loan amount below the configured minimum.
- Tenure below 1 year or above 30 years.
- Employment duration below 0.5 years.

Non-Functional Testing

- Responsiveness
- Usability
- Error handling
- Security of API keys
- Performance of client-side calculations

Deployment

After completing the website development, all the required project files, including the HTML, CSS, JavaScript, configuration, and other supporting files, were uploaded to a GitHub repository for version control and project management.

The GitHub repository was then connected to Vercel, a cloud-based deployment platform. The GitHub repository link was provided to Vercel, which imported the project files and deployed the website. After successful deployment, Vercel generated a public website URL through which the FinWise AI application could be accessed online.

Execution

After deployment, the FinWise AI website can be accessed directly through its official Vercel URL using a web browser. Users can open the website and interact with its different modules, including the Home Page, Loan Eligibility Checker, Credit Score Analyzer, EMI Calculator, AI Financial Tips, and Dashboard.

The deployed website was tested by opening the official URL and navigating through the available features. Users can enter financial information, perform calculations, analyze credit-related information, and interact with the available financial tools directly through the online website without running the project files manually on their local computer.

Conclusion

FinWise AI combines loan eligibility assessment, credit-score analysis, EMI calculation, AI financial guidance and dashboard visualization into one responsive BFSI web interface. Its modular design allows users to move from financial-data entry to calculation, interpretation and review without switching between separate tools.

The project demonstrates how frontend interaction, financial formulas, API-based AI assistance and data-storage integration can be combined in a practical academic web application. The current supplied source material strongly supports the frontend modules and documented architecture; backend implementation details should be completed or verified before treating the system as production-ready.